

NLP

Term Project: LLM Classification  
Finetuning

Group 46: 林鈺承 林俞琛 徐綉惠

# 1. Introduction

Large language models (LLMs) are increasingly integrated into various aspects of our daily lives, providing powerful tools for communication and interaction. However, ensuring that their responses align with user preferences is critical for fostering effective and meaningful interactions. This competition offers a unique opportunity to address this challenge by utilizing real-world data to better understand and predict user preferences in LLM-generated responses.

The dataset used in this competition is derived from the Chatbot Arena, where users engage in head-to-head conversations with two anonymous LLMs and choose their preferred response. The goal of this competition is to develop a model that accurately predicts which response a user will favor in these pairwise comparisons. The dataset consists of 55,000 training samples, each including a prompt, two different responses generated by two different models, and a truth label indicating which response is better. The testing dataset contains 25,000 samples, each including a prompt and two responses that are hidden on the Kaggle platform. We can only submit the training code to evaluate our performance. It is worth noting that while the model names are provided in the training set, they are not included in the testing set.

This task aligns with the concept of "reward models" or "preference models" in reinforcement learning from human feedback (RLHF). Previous studies have highlighted the challenges in directly prompting an LLM to predict user preferences, with biases such as position bias (favoring the first response), verbosity bias (favoring more verbose answers), and self-enhancement bias (favoring responses with self-promoting content) often influencing outcomes.

In this project, we aim to focus on a **multi-task model framework** to leverage the model names by first calculating the win rate of each model and then utilizing this win rate information to enhance performance. Additionally, we aim to **apply data augmentation methods** to prevent overfitting during training and enable the model to extract more information from the training data.

## 2. Multi-task Model Framework

- What method we picked and why

When we were digging through reference materials, we found that some people had put together and analyzed win rates between different models using various charts. This sparked an idea for us—maybe we could focus on model win rates. After all, the training data includes the model names, so we could calculate the win rates myself and incorporate them into the training as one of the weights.

But soon we hit our first obstruction: the validation data doesn't include model names. At that point, we had two options. One was to train the model to learn how to identify which model generated a given sentence. That way, even without the model names in the validation data, my model could still figure it out and pull up the corresponding win rates. The other option was to use multitask learning, where the model would combine the loss from both the original labels and the win rate labels during training. Essentially, this means letting the win rates influence the parameter updates, and then turning off the win rate input during validation. The second option seemed more viable, so we ultimately decided to go with multitask learning.

- How we implemented the multi-task framework
  1. Calculate the win rates between each pair of models, including three probabilities: model A wins, model B wins, and a tie.
  2. Add these win rate stats to the original dataset by inserting three new columns.
  3. Filter out any data where the win rate is above 0.5, since that indicates there aren't enough samples to reliably calculate those win rates.
  4. Add a regression layer to the existing TinyBERT model and combine the losses from both outputs.

### 3. Data Truncation and Augmentation

Our framework for data preprocessing consists of using the TinyBERT tokenizer to tokenize the training data into tokens and attention masks. However, since we are using the TinyBERT model and tokenizer, we are limited to a maximum of 512 tokens, which means that any data exceeding 512 tokens will have its tail truncated. This would waste useful information in data that contains long paragraphs of words.

Because of this, we implemented a **random truncation algorithm that randomly selects a contiguous section of the data containing 512 words and tokenizes it**. This means that the model can learn from data beyond the first 512 words and extract more information. In addition, we also randomly switched the positions of response\_a and response\_b to ensure that the model doesn't rely on making decisions based on the order of the responses.

- |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>● <b>Input:</b> A text (prompt or response) along with a specified maximum length for the text. ∙</li><li>● <b>Operation:</b> Randomly truncates the text by: ∙<br/>Step 1. Divide the text into individual words. ∙<br/>Step 2. Ensure that the text does not exceed the maximum length. ∙<br/>Step 3. Randomly select a contiguous subset of words, ensuring that it does not exceed a specified maximum length if the text is excessively long. ∙</li><li>● <b>Output:</b> The truncated text. ∙</li></ul> |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Table. 1 Random text truncation algorithm for data pre-processing.

Originally, we concatenated the prompt and response and then truncated the result to be tokenized. This meant that we had two inputs: “prompt + response\_a” and “prompt + response\_b,” both being 512 tokens long. By concatenating the text first, we hoped the model could learn the context between the prompt and response. However, after further experiments, we found a better method: **first tokenizing “prompt,” “response\_a,” and “response\_b” separately, then concatenating the embeddings after passing them through the TinyBERT embedding model**. With this method, there are three sets of tokens, each 512 tokens long, which contain more text for the model to train on.

As a data augmentation method, we also tried random data deletion, where random words are deleted with a certain probability.

After pre-processing, data augmentation, and tokenization, the tokens and attention masks are sent to the model for training.

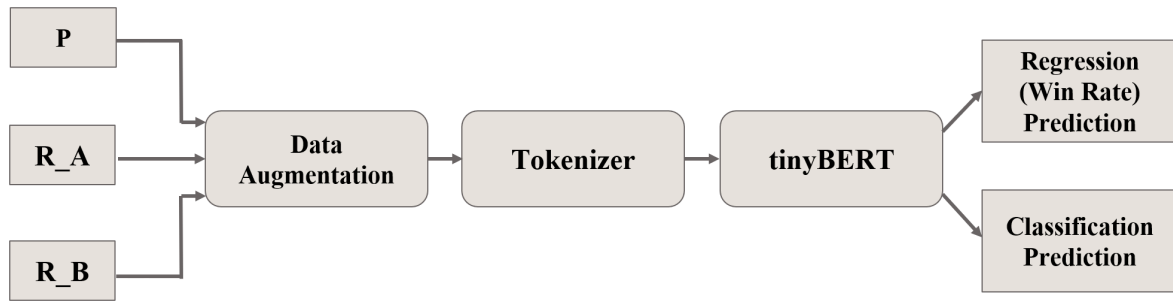


Fig. 1 Overall framework.

## 4. Experiments and Results

### (a) Multi-task Model

The multi-task model consists of a classification loss and a regression loss. In this experiment, we fixed the classification loss weight to 1 and varied the weight of the regression loss to observe its impact on the model's performance. From the table below, we can see that **the model performs best when the regression loss weight is set to 5.**

| Model      | Regression Loss Weight | Validation Accuracy | Validation F1 Score |
|------------|------------------------|---------------------|---------------------|
| Multi-task | 10                     | 0.4713              | 0.4572              |
| Multi-task | 5                      | 0.4726              | 0.4684              |
| Multi-task | 1                      | 0.4654              | 0.4559              |
| Multi-task | 0.1                    | 0.4683              | 0.4629              |

Table. 2 Validation results using different regression loss weights for the multi-task model.

### (b) Random Truncation

Here, we compare the validation accuracy and F1 score results when using our random truncation algorithm. As shown in Table. 3, when we first concatenate “prompt” and “response” and then send them to the tokenizer, the validation accuracy and F1 score are the lowest. However, **when we first tokenize “prompt,” “response\_a,” and “response\_b” separately and then concatenate them after the TinyBERT embedding model, the validation accuracy increased to 0.4763.** Finally, when we applied random truncation, the validation accuracy further increased to 0.4828, and the F1 score increased to 0.4711.

| Model                    | Tokenized Text                                  | Random Truncation | Validation Accuracy | Validation F1 Score |
|--------------------------|-------------------------------------------------|-------------------|---------------------|---------------------|
| TinyBERT (No Multi-task) | “prompt + response_a”,<br>“prompt + response_b” | No                | 0.4548              | 0.4346              |
| TinyBERT (No Multi-task) | “prompt”,<br>“response_a”,<br>“response_b”      | No                | 0.4763              | 0.4644              |
| TinyBERT (No Multi-task) | “prompt”,<br>“response_a”,<br>“response_b”      | Yes               | 0.4828              | 0.4711              |

Table. 3 Validation results using different tokenization strategies with and without random truncation.

### (c) Data Augmentation

Here we compare the validation accuracy of different data augmenting methods. We can tell that **the model performed the best when random deletion and insertion rate is both 0.1.**

| Random Deletion | Random Insertion | Validation Accuracy | Validation F1 Score |
|-----------------|------------------|---------------------|---------------------|
| 0               | 0                | 0.4667              | 0.4560              |
| 0.1             | 0.1              | 0.4711              | 0.4635              |
| 0.4             | 0.4              | 0.4700              | 0.4641              |

Table. 4 Validation results using different random deletion and insertion rates.

### (d) Discussion

From experiments (a) and (b) above, we can see that the results without using the multi-task model are better than expected. This means that by purely applying random truncation to the text, we achieved the best performance with a validation accuracy of 0.4828. We believe there are several possible reasons for this outcome.

First, when using the multi-task model framework, the model needs to learn to predict both the win rate and the better response. This inherently **increases the amount of information the model needs to learn, much of which is not utilized during the testing phase.** Consequently, **this could interfere with the model's ability to focus on the original classification task, leading to confusion during training.** Causing the accuracy to decrease.

Second, the way the model learns to predict the win rate might not contribute to improving performance in the classification task as we initially hoped. While the multi-task model learns to predict the win rate, **there is no direct connection between the two tasks.** Even if the model accurately predicts the win rate, there is no strong reason to assume that this would also enhance its performance in the classification task.

In the end, the multi-task model did not meet our expectations, and the best validation result was achieved using the TinyBERT classification model with the random text truncation, with a validation accuracy of 0.4828. This model also performed the best when submitted to Kaggle.

### (e) Kaggle Leaderboard Score

Through these three experiments, we found out the best parameters for the best performance in each of the methods. Then what we did next was to combine them together in the submission of the kaggle competition.

The best results we got in Kaggle is using the TinyBERT classification model with the random text truncation. **In the end, we got 65th place at the time when writing this report with a score of 1.04387.**

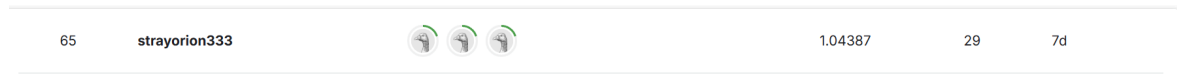


Fig. 2 Best Kaggle score when writing the report.

## 5. Conclusion and Future Work

The most challenging part of this project was navigating the limitations on model size and training time imposed by the competition rules. However, we managed to devise methods to address these constraints. By applying random truncation to the training data, we enabled the model to learn more information and achieve better performance. **Finally, we got 65th place in the Kaggle competition at the time when writing this report with a score of 1.04387.**

Although the multi-task model did not perform as expected, utilizing the win rate in the training data remains key to further improving the model. In the future, we plan to make the regression loss that predicts the win rate influence the classification process more directly, allowing the model to better leverage the information in the win rate data. Additionally, we will explore using backbone models beyond TinyBERT or perhaps even entirely different frameworks. By incorporating more complex models and frameworks, the model may learn the information more effectively.